Cuaderno de campo

Simulaciones computacionales para la visualización y análisis de problemas clásicos de física
Semillero MATBIOM, Universidad de Medellín
EISI RedCOLSI Panamá 2026

Datos generales

Título del proyecto	Simulaciones computacionales para la visualización y análisis de problemas clásicos de física
Semillero	MATBIOM
Institución	Universidad de Medellín
Programa académico	Computación Científica
Docente asesor	Hernán Alejandro Gómez Urrea
Integrantes	Daniel Santiago Machado Serna, Felipe Alonso Obando, Samuel David Gallego Meneses, Juliana Franco, Emanuel Orozco, André Rivas, Felipe Villa Jaramillo
Fecha de inicio	31/03/2025
Estado actual	Proyecto en curso.
Repositorio / enlace del proyecto	https://github.com/sanma613/matbiom-lab-virtual-fisica

Resumen

El proyecto desarrolla un laboratorio virtual de física con cinco experimentos de mecánica clásica: tiro parabólico, sistema masa-resorte, péndulo simple, péndulo doble y colisiones en dos dimensiones. Responde a dos problemas: el acceso limitado a laboratorios físicos y la dependencia del acompañamiento docente para interpretar fenómenos abstractos. El backend está escrito en Python con FastAPI. Allí, motores numéricos, Verlet de velocidad y Runge-Kutta de orden 4, integran las ecuaciones de movimiento con un paso fijo de 1/60 s y envían el estado en tiempo real por WebSocket. El frontend, hecho en Next.js, anima cada sistema en un lienzo Canvas y grafica posición, velocidad y energía. El estudiante puede modificar los parámetros físicos, pausar la simulación y registrar mediciones en una tabla de laboratorio. Cada experimento trae una guía con objetivos, marco teórico, procedimiento y preguntas. El estudiante redacta hipótesis, observaciones y conclusiones en un cuaderno digital. Un tutor de inteligencia artificial, que usa un modelo de lenguaje servido por Groq, revisa el reporte y formula preguntas socráticas ancladas en los datos, sin entregar la respuesta. Hasta ahora se tiene una versión funcional con los cinco experimentos. Un script de verificación automática con

20 criterios, que compara cada simulación con soluciones analíticas y con la conservación de energía y momento, se supera en su totalidad.
Palabras clave: laboratorio virtual, simulación computacional, mecánica clásica, métodos numéricos, tutor de inteligencia artificial

Planteamiento del problema

Contexto y situación problema

La enseñanza tradicional de la física suele carecer de entornos interactivos para explorar conceptos abstractos. A esto se suman las limitaciones del laboratorio tradicional: acceso restringido por costos de equipos, disponibilidad de espacios y horarios, e imposibilidad de variar libremente la gravedad, eliminar la fricción o repetir con exactitud una condición inicial caótica como la del péndulo doble. La computación científica ofrece un puente clave para explorar fenómenos complejos (Schecker, 1993), pero interpretar fenómenos como la conservación de la energía, la no linealidad o el caos determinista sigue dependiendo del acompañamiento continuo del docente, lo que limita el aprendizaje autónomo.

Pregunta de investigación

¿De qué manera el diseño y uso de entornos computacionales interactivos favorece la comprensión conceptual y el análisis autónomo de problemas clásicos de física?

Justificación

La enseñanza tradicional suele carecer de entornos interactivos para explorar conceptos abstractos. Las simulaciones computacionales superan esta limitación (Schecker, 1993) al ofrecer representaciones visuales dinámicas que permiten manipular variables clave y analizar su respuesta en tiempo real, promoviendo la experimentación y el aprendizaje autónomo. Los beneficiarios directos son los estudiantes de los cursos de física de la Universidad de Medellín y los docentes que los orientan. Al ser de código abierto, el proyecto también puede servir a otras instituciones con recursos de laboratorio limitados.

Objetivos

Objetivo general

Desarrollar un entorno de simulaciones computacionales interactivas en Python para el modelado, visualización y análisis de problemas clásicos de mecánica.

Objetivos específicos

1. Proporcionar una herramienta educativa que facilite la comprensión de conceptos abstractos de física.
2. Analizar los resultados de las simulaciones y compararlos con los modelos teóricos.
3. Fomentar el interés y la motivación de los estudiantes por la física a través de la exploración y la experimentación virtual.

Hipótesis

El proyecto plantea la siguiente hipótesis:
Un entorno de simulaciones interactivas, con manipulación de parámetros, mesa de laboratorio y tutor de IA, construido sobre métodos numéricos validados favorece la comprensión conceptual y el análisis autónomo de problemas clásicos de física. Los métodos Verlet y RK4 reproducen las soluciones analíticas y conservan los invariantes físicos dentro de las tolerancias establecidas.

Variables

La clasificación se revisó con base en la implementación actual.

Tipo	Variable	Descripción / unidad
Independientes	Condiciones iniciales y parámetros físicos	Ángulos, velocidades iniciales, masas, longitudes, constante del resorte, coeficiente de amortiguamiento o arrastre, coeficiente de restitución, gravedad, paso de tiempo
Dependientes	Magnitudes medidas en la simulación	Posición, velocidad, período, energía cinética, potencial y total, momento lineal
Dependientes de verificación	Métricas de error	RMSE frente a la solución analítica, error del período, deriva relativa de energía
Controladas	Método de integración y tolerancias	Verlet o RK4 según el experimento, umbral de cada verificación
Controladas	Paso de tiempo y duración	Paso fijo $\Delta t = 1/60$ s, con subpasos adaptativos en el resorte y el péndulo doble; duración máxima de 90 s por ejecución

Marco teórico

Simulación computacional en la enseñanza de la física

Las simulaciones computacionales superan las limitaciones de la enseñanza tradicional al ofrecer representaciones visuales dinámicas que permiten manipular variables clave y analizar su respuesta en tiempo real, promoviendo la experimentación y el aprendizaje autónomo (Schecker, 1993). Mediante herramientas interactivas en Python (Python Software Foundation, 2024), los problemas clásicos de la física se modelan y visualizan para transformar conceptos

abstractos en intuición visual y numérica. El proyecto aplica física computacional a problemas clásicos de mecánica (Young y Freedman; Alonso y Finn) y valida los modelos mediante métodos numéricos (Giordano y Nakanishi, 2005), contrastando las trayectorias con soluciones analíticas o evaluando la conservación de la energía en los casos aplicables.

Mecánica clásica de los sistemas estudiados

- **Proyectil:** cinemática con y sin arrastre lineal, con fuerza F = −b·v. Sin arrastre, y con arrastre, respectivamente:

$$x(t) = x_0 + v_{0x}t, \quad y(t) = y_0 + v_{0y}t - \tfrac{1}{2}gt^2$$

$$v_x(t) = v_{0x}e^{-bt/m}, \quad v_y(t) = \left(v_{0y} + \tfrac{mg}{b}\right)e^{-bt/m} - \tfrac{mg}{b}$$

$$x(t) = \tfrac{m}{b}v_{0x}(1 - e^{-bt/m}), \quad y(t) = \tfrac{m}{b}\left(v_{0y} + \tfrac{mg}{b}\right)(1 - e^{-bt/m}) - \tfrac{mg}{b}t$$

- **Oscilador masa-resorte:** movimiento armónico simple y subamortiguado. En el régimen subamortiguado:

$$m\ddot{x} + c\dot{x} + kx = 0$$

$$\omega_0 = \sqrt{k/m}, \quad \gamma = \tfrac{c}{2m}, \quad \omega_d = \sqrt{\omega_0^2 - \gamma^2}$$

$$x(t) = Ae^{-\gamma t}\cos(\omega_d t + \varphi)$$

- **Péndulo simple:** período exacto mediante integral elíptica para grandes ángulos, y aproximación para ángulos pequeños:

$$\ddot{\theta} + \tfrac{g}{L}\sin\theta = 0$$

$$T = 4\sqrt{\tfrac{L}{g}}\,K(k), \quad k = \sin\tfrac{\theta_0}{2}$$

$$T \approx 2\pi\sqrt{L/g} \quad (\theta_0 \ll 1)$$

- **Péndulo doble:** sistema no lineal y caótico. Las dos ecuaciones acopladas para θ_1 y θ_2 se obtienen de las ecuaciones de Euler-Lagrange del lagrangiano; la energía total debe conservarse:

$$\mathcal{L} = T - V, \qquad E = T + V$$

- **Colisiones:** conservación de momento y coeficiente de restitución e, donde e = 1 es una colisión elástica y e = 0 una perfectamente inelástica:

$$m_A v_A + m_B v_B = m_A v'_A + m_B v'_B$$

$$e = -\frac{(\mathbf{v}'_B - \mathbf{v}'_A)\cdot\hat{n}}{(\mathbf{v}_B - \mathbf{v}_A)\cdot\hat{n}}$$

Métodos numéricos

Verlet de velocidad: integrador simplético de segundo orden. Calcula la posición con la aceleración actual y corrige la velocidad con el promedio de las aceleraciones inicial y final. Se usa en el proyectil y el resorte porque mantiene acotado el error de energía en simulaciones largas. Runge-Kutta de orden 4, abreviado RK4: método de cuarto orden con error global $O(\Delta t^4)$, usado en el péndulo simple y el péndulo doble, formulados como sistemas de EDO. En el resorte y el péndulo doble, el número de subpasos por cuadro se ajusta según la frecuencia natural del sistema para mantener la precisión. Las colisiones se resuelven mediante impulsos con coeficiente de restitución. La calidad numérica se evalúa con el RMSE frente a la solución analítica, el error del período y la deriva relativa de la energía total. Fórmulas de las métricas:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(x_i^{\mathrm{num}} - x_i^{\mathrm{an}}\right)^2}$$

$$\delta E(t) = \frac{|E(t) - E_0|}{|E_0|}, \qquad \varepsilon_{\mathrm{RK4}} = \mathcal{O}(\Delta t^4)$$

Tutor de inteligencia artificial y aprendizaje autónomo

La retroalimentación guiada se basa en el método socrático: en lugar de dar la respuesta, el tutor formula preguntas que llevan al estudiante a construir su propia explicación. El tutor usa un modelo de lenguaje servido por la API de Groq, con openai/gpt-oss-20b como modelo por defecto y modelos alternativos configurables. En cada consulta recibe el estado actual del experimento: posición, velocidad, energía cinética, potencial y total, su tendencia y los parámetros físicos. Un mensaje de sistema con directivas obligatorias impide responder temas ajenos a la física, hacer cálculos por el estudiante y emitir veredictos como "correcto" o "incorrecto". Además, lo obliga a citar un valor numérico del estado actual y a responder en máximo dos oraciones, con notación LaTeX. El tutor también revisa el cuaderno del estudiante, es decir, hipótesis, observaciones, datos y conclusiones, y devuelve una retroalimentación orientadora.

Materiales y recursos utilizados

Recurso	**Uso en el proyecto**
Python	Backend y motores de simulación
FastAPI	API del laboratorio virtual
Next.js	Interfaz web del laboratorio
NumPy y SciPy	Cálculo numérico y verificación de la integral elíptica
Tutor de IA	Modelo de lenguaje vía API de Groq, con openai/gpt-oss-20b por defecto
Equipos	Equipo de cómputo con procesador multinúcleo x86_64, 16 GB RAM, sistema operativo Linux Ubuntu, entorno Node.js v20+, Python 3.12 y navegador web con soporte HTML5 Canvas y WebSockets.
Control de versiones	Git y GitHub en github.com/sanma613/matbiom-lab-virtual-fisica)
React, KaTeX y marked	Componentes de la interfaz y visualización de texto Markdown con ecuaciones LaTeX

Procedimiento

La metodología sigue cuatro fases presentadas en el póster: selección de sistemas, modelado numérico, visualización dinámica y validación técnica. Estas fases se concretaron en las siguientes etapas:



Fases metodológicas del proyecto, tomadas del póster científico.

1. Revisión bibliográfica y selección de los problemas clásicos. Se seleccionaron cinco sistemas representativos de la mecánica clásica con solución analítica o invariantes conocidos, lo que permite validar cada simulación.
2. Modelado físico y elección del método numérico para cada experimento. Se plantearon las ecuaciones de movimiento de cada sistema. Se eligió Verlet de velocidad para los modelos basados en fuerzas, como proyectil y resorte, RK4 para los modelos formulados como EDO, como los péndulos, y un esquema por impulsos para las colisiones.
3. Implementación de los motores de simulación en Python. Se construyó un núcleo modular: las clases State y Vec2, una interfaz ForceModel, un ODEModel abstracto, los motores VerletEngine y RK4Engine, y un módulo por experimento. Una fábrica de motores crea cada simulación a partir de los parámetros recibidos y los limita a rangos físicos seguros.
4. Desarrollo de la API en FastAPI y de la interfaz en Next.js. Se implementó un WebSocket en la ruta /ws que avanza la simulación con paso fijo de 1/60 s y envía cada estado en formato JSON, con comandos de pausa y reanudación. La interfaz en Next.js anima el sistema en un Canvas, grafica posición, velocidad y energía, y ofrece controles para los parámetros.
5. Desarrollo de la mesa de laboratorio: tabla de datos, captura de mediciones y exportación. Se añadieron guías de laboratorio por experimento y un cuaderno digital donde el estudiante registra hipótesis, observaciones, una tabla de mediciones tomadas de la simulación y conclusiones.
6. Integración del tutor de IA para la revisión de reportes. Se crearon los servicios /api/tutor para preguntas socráticas durante el experimento y /api/tutor/review para la revisión del cuaderno, conectados a un modelo de lenguaje de Groq con directivas pedagógicas estrictas.
7. Verificación automática contra soluciones analíticas e invariantes físicos. Se escribió un script que ejecuta 20 pruebas: compara cada simulación con su solución analítica o con

la conservación de energía y momento, contra umbrales definidos. Se generaron figuras comparativas para el póster.

Bitácora de actividades por fecha

Fecha	Actividad realizada	Evidencia / soporte	Observaciones y decisiones
Fase 1: Fundamentación teórica, formación y revisión bibliográfica, marzo a julio de 2025			
31/03/2025	Inicio oficial del semillero MATBIOM y sesión introductoria: formulación del problema sobre enseñanza de la física, limitaciones de laboratorios reales y pertinencia de la computación científica.	Acta de inicio del semillero / Presentación institucional	Se definieron las líneas de trabajo y se acordó realizar sesiones formativas semanales todos los viernes.
11/04/2025	Sesión formativa recurrente de los viernes: fundamentación en mecánica newtoniana y formulación lagrangiana para sistemas conservativos y disipativos.	Apuntes teóricos y bibliografía de Young & Freedman y Alonso & Finn	Se repasaron formalismos matemáticos para describir cinemática y dinámica en sistemas de 1 y 2 grados de libertad.
09/05/2025	Sesión formativa recurrente de los viernes: estudio de algoritmos de integración numérica para EDOs aplicadas a la física como Euler, Verlet de velocidad y Runge-Kutta 4.	Referencias teóricas (Giordano & Nakanishi, 2005)	Se identificó que métodos simplécticos como Verlet preservan mejor los invariantes en sistemas conservativos, mientras RK4 es idóneo para EDOs de orden superior.
06/06/2025	Sesión formativa recurrente de los viernes: análisis de dinámicas no lineales, espacio de fases y caos determinista en sistemas mecánicos acoplados.	Modelos analíticos de péndulo doble	Se seleccionó el péndulo doble como caso de estudio paradigmático de alta no linealidad y sensibilidad a condiciones iniciales.
04/07/2025	Sesión de cierre de la etapa formativa del viernes previo al 07/07/2025: consolidación de conceptos y selección preliminar de los cinco problemas clásicos a modelar.	Documento de síntesis y temáticas seleccionadas	Se concluyó la etapa de fundamentación y orientación teórica para dar paso a la futura fase de implementación computacional.
Fase 2: Diseño arquitectónico y modelado físico de simulaciones, abril a mayo de 2026			
10/04/2026	Inicio de la fase de implementación de código: definición de la arquitectura cliente-servidor con backend en FastAPI	Repositorio GitHub en github.com/sanma613/ matbiom-lab-virtual-	Se optó por centralizar el cálculo numérico en Python mediante

Fecha	Actividad realizada	Evidencia / soporte	Observaciones y decisiones
	y frontend en Next.js e inicialización del repositorio.	fisica)	streaming por WebSocket para garantizar fidelidad física independiente del cliente.
24/04/2026	Modelado e implementación del motor de tiro parabólico en 2D con y sin fuerza de arrastre lineal proporcional a la velocidad.	Archivo backend/app/experiments/projectile.py	Se empleó el integrador Verlet de velocidad. El arrastre se parametrizó con el coeficiente drag_coeff.
08/05/2026	Modelado e implementación del oscilador armónico y amortiguado, es decir, el sistema masa-resorte con ley de Hooke y amortiguamiento viscoso.	Archivo backend/app/experiments/spring.py	Se incorporaron subpasos adaptativos automáticos en función de $\omega_0 = \sqrt{(k/m)}$ y detección de colisión elástica con el suelo.
22/05/2026	Modelado e implementación del péndulo simple para grandes oscilaciones angulares, sin linealización de ángulo pequeño.	Archivo backend/app/experiments/pendulum.py	Se formuló como sistema de EDO de primer orden integrado con RK4 para contrastar el período contra la integral elíptica exacta.
Fase 3: Modelado avanzado y desarrollo del frontend interactivo, junio a julio de 2026			
05/06/2026	Modelado e implementación del péndulo doble caótico mediante ecuaciones de Euler-Lagrange acopladas.	Archivo backend/app/experiments/double_pendulum.py	Se aplicó RK4 con subpasos adaptativos calibrados por la velocidad angular máxima de ambas barras para evitar inestabilidades numéricas.
19/06/2026	Modelado e implementación del sistema de colisiones bidimensionales entre dos discos rígidos con gravedad y arrastre opcionales.	Archivo backend/app/experiments/collision.py	Se resolvió mediante el método de impulsos instantáneos a lo largo de la normal de contacto con coeficiente de restitución $e \in [0, 1]$.
03/07/2026	Desarrollo de la interfaz gráfica interactiva en Next.js con renderizado dinámico en HTML Canvas y conexión WebSocket a 60 Hz.	Componentes SimulationCanvas.jsx y CanvasRenderer.jsx	Se implementó un lienzo reactivo que dibuja trayectorias, vectores y cuerpos con escalado automático de coordenadas físicas a píxeles.

Fecha	Actividad realizada	Evidencia / soporte	Observaciones y decisiones
24/07/2026	Implementación del panel de control de parámetros en tiempo real, gráficas dinámicas de energía y cinemática, y comandos de pausa/reanudación.	Componentes Controls.jsx y Charts.jsx	Se permitió a los estudiantes pausar la simulación en instantes clave para tomar mediciones precisas de posición y velocidad.
Fase 4: Retroalimentación docente, guías y mesa de laboratorio, agosto de 2026			
20/08/2026	Reunión virtual con el docente asesor Hernán Alejandro Gómez Urrea para revisión técnica de la plataforma y retroalimentación pedagógica sobre la interfaz.	Acuerdos de reunión y observaciones del docente	Se acordó incorporar una mesa de trabajo estructurada y guías formales de laboratorio para que la herramienta no fuera solo visualizador sino un entorno de indagación guiada.
21/08/2026	Recepción, revisión y análisis de las guías de laboratorio de física suministradas por el profesor Hernán Gómez para integrarlas a la plataforma.	Guías de laboratorio docentes de referencia	Se tomó como base la estructura pedagógica de las guías docentes: objetivos, fundamentación teórica, procedimiento experimental y preguntas orientadoras).
28/08/2026	Implementación de la mesa de laboratorio virtual y cuaderno de campo digital del estudiante en LabNotebook.jsx y labGuides.js.	Código de LabNotebook.jsx y labGuides.js	Se programó una tabla dinámica de toma de datos experimentales, campos de texto para hipótesis, observaciones y conclusiones con renderizado Markdown y KaTeX.
Fase 5: Tutor de IA socrático y verificación física rigurosa, septiembre de 2026			
04/09/2026	Integración del tutor inteligente socrático mediante la API de Groq y desarrollo del endpoint de revisión pedagógica de cuadernos de laboratorio.	Endpoint backend/app/api/tutor.py y TutorPanel.jsx	Se redactó un System Prompt con directivas deterministas estrictas: citar valores numéricos del experimento, jamás emitir veredictos ni dar la respuesta matemática.
18/09/2026	Diseño y codificación del script de verificación física y numérica	Script verify_physics.py	Se implementaron comparaciones contra

Fecha	Actividad realizada	Evidencia / soporte	Observaciones y decisiones
	automatizada con 19 criterios iniciales objetivos de tolerancia.		soluciones analíticas exactas como alcance, integral elíptica para el período del péndulo y conservación de energía y momento.
25/09/2026	Calibración y corrección de métricas experimentales en pruebas del resorte, con la tasa de decaimiento γ, y colisiones, con la restitución empírica medida; validación de 20 de 20 criterios superados.	Salida de ejecución del script verify_physics.py con 20/20 PASS	Se ajustaron las pruebas para que calcularan las variables directamente desde la serie de datos simulados, asegurando rigor metodológico total.
Fase 6: Socialización, divulgación y documentación final, octubre de 2026			
02/10/2026	Generación y exportación de gráficos científicos vectoriales de validación y trayectorias en formato PDF y SVG para divulgación.	Script plot_physics.py y figuras en carpeta poster_figs/	Se generaron las 5 figuras científicas que contrastan trayectorias simuladas contra soluciones analíticas.
09/10/2026	Diseño, redacción y maquetación del póster científico para el Encuentro Internacional de Semilleros de Investigación, EISI RedCOLSI Panamá 2026.	Archivo de presentación Poster Semillero MATBIOM.pptx	Se sintetizaron introducción, antecedentes, metodología en 4 pasos, resultados de verificación gráfica y conclusiones.
17/10/2026	Consolidación final del repositorio de código abierto en GitHub con README técnico en inglés, actualización del cuaderno de campo y registro de evidencias del semillero.	Commit en repositorio GitHub y cuaderno de campo diligenciado	Se documentaron instrucciones completas de instalación, contratos de WebSocket, endpoints del tutor y sustento teórico del proyecto.

Resultados

Estado del laboratorio virtual

Primera versión funcional con cinco experimentos operativos, dinámicos y en tiempo real, mesa de laboratorio con tabla de datos y reporte, y tutor de IA integrado.

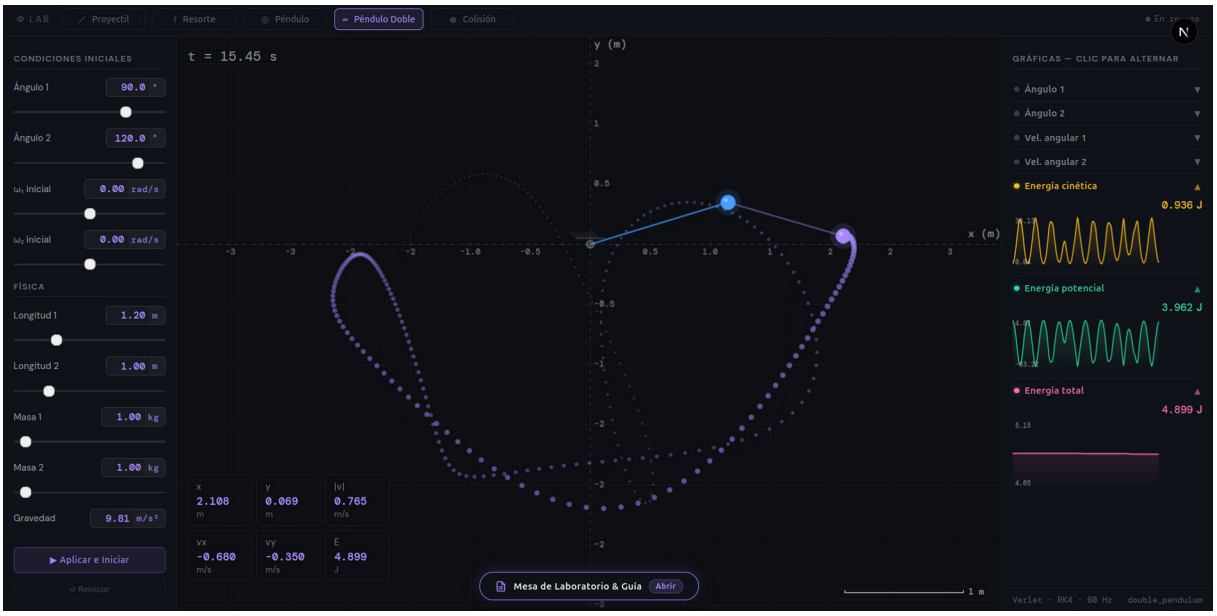
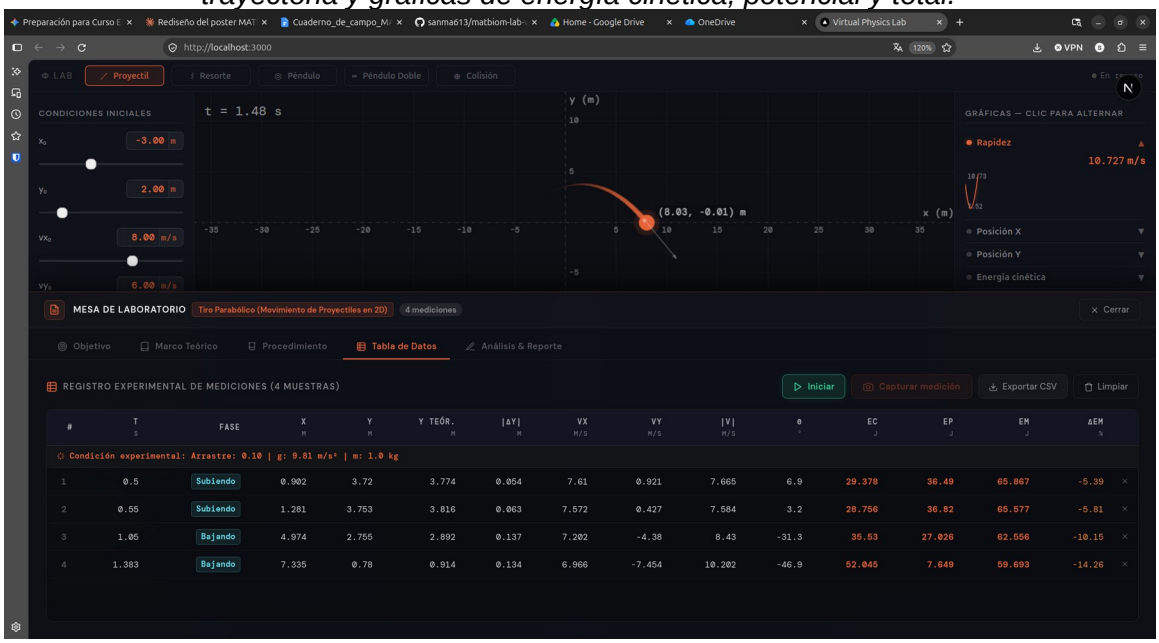


Figura 1. Interfaz del laboratorio virtual: péndulo doble en tiempo real con condiciones iniciales, trayectoria y gráficas de energía cinética, potencial y total.



#	T (s)	FASE	X (m)	Y (m)	Y TEÓR. (m)	|ΔY| (m)	VX (m/s)	VY (m/s)	|V| (m/s)	θ (°)	EC (J)	EP (J)	EM (J)	ΔEM (%)
1	0.5	Subiendo	0.902	3.72	3.774	0.054	7.61	0.921	7.665	6.9	29.378	36.49	65.867	-5.39
2	0.55	Subiendo	1.281	3.753	3.816	0.063	7.572	0.427	7.584	3.2	28.756	36.82	65.577	-5.81
3	1.05	Bajando	4.974	2.755	2.892	0.137	7.202	-4.38	8.43	-31.3	35.53	27.026	62.556	-10.15
4	1.383	Bajando	7.335	0.78	0.914	0.134	6.966	-7.454	10.202	-46.9	52.045	7.649	59.693	-14.26

Figura 2. Mesa de laboratorio del tiro parabólico: tabla de mediciones con Y experimental frente a Y teórico, energías y condición experimental de arrastre 0.10.

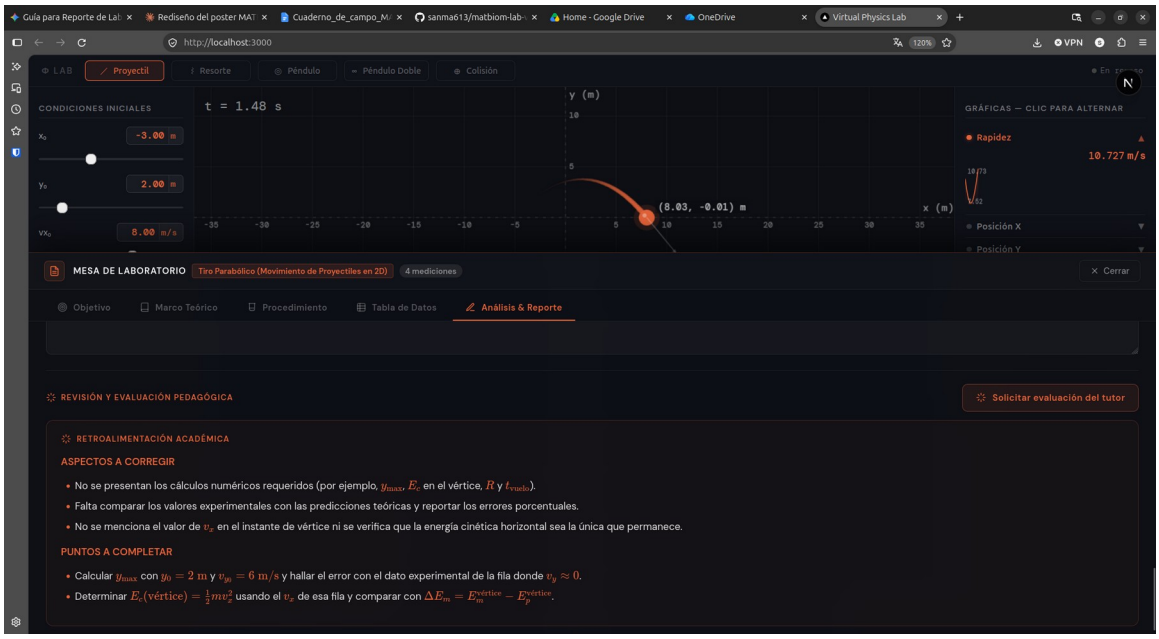


Figura 3. Retroalimentación académica del tutor de IA: señala aspectos a corregir y puntos a completar sin dar la respuesta.

Verificación física y numérica

Se ejecutó un script de verificación automática con 20 criterios. Todos fueron superados. Cada simulación se compara con su solución analítica o con la conservación de energía y momento.

Experimento	**Criterios**	**Qué se verificó**	**Resultado destacado**
Proyectil	5	Posición sin arrastre y con arrastre lineal frente a solución analítica; tiempo de vuelo	RMSE con arrastre de 4.0×10^{-3} m en X y 1.1×10^{-2} m en Y
Resorte	4	Oscilación libre y amortiguada frente a solución analítica; energía simulada no crece; tasa de decaimiento	y analítico 0.2000 y y medido 0.1996
Péndulo simple	5	Período frente a integral elíptica a 10°, 45°, 90° y 150°; conservación de energía a 80°	Error del período inferior a 4×10^{-9} s
Péndulo doble	3	Conservación de energía en 15 s; caso límite que reduce al	Deriva relativa de energía de 4.3×10^{-7}

		péndulo simple; energía no crece con fricción	
Colisiones	3	Conservación de momento y energía en choque elástico; coeficiente de restitución medido	Restitución medida de 0.600 para e = 0.6

Fuente: salida del script de verificación verify_physics.py. La salida completa se incluye en el Anexo A.

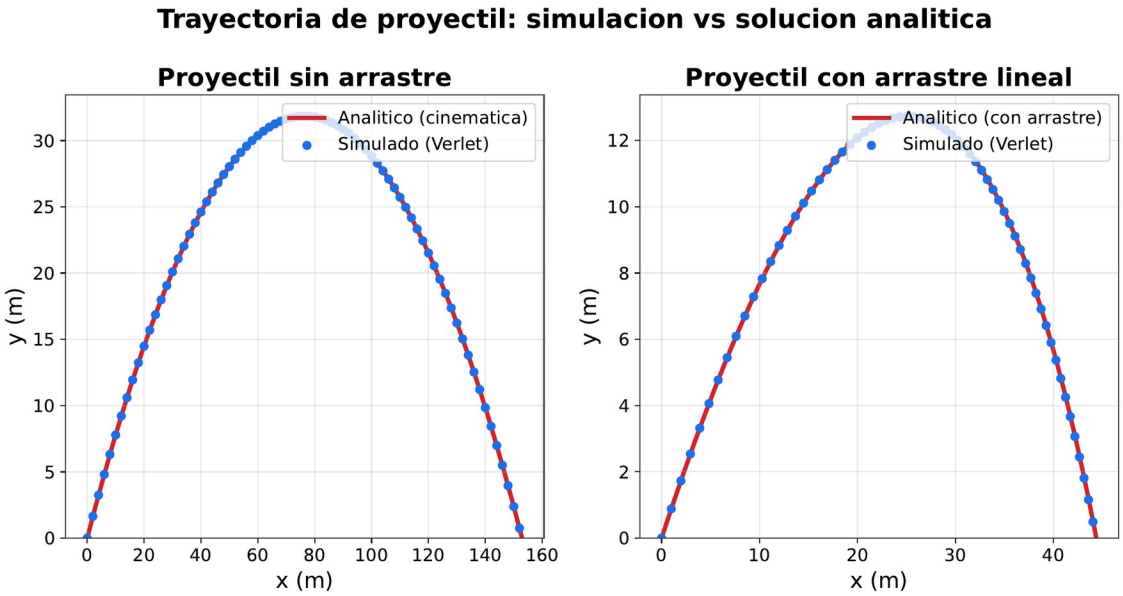


Figura 4. Trayectoria del proyectil: simulación frente a solución analítica, con y sin arrastre.

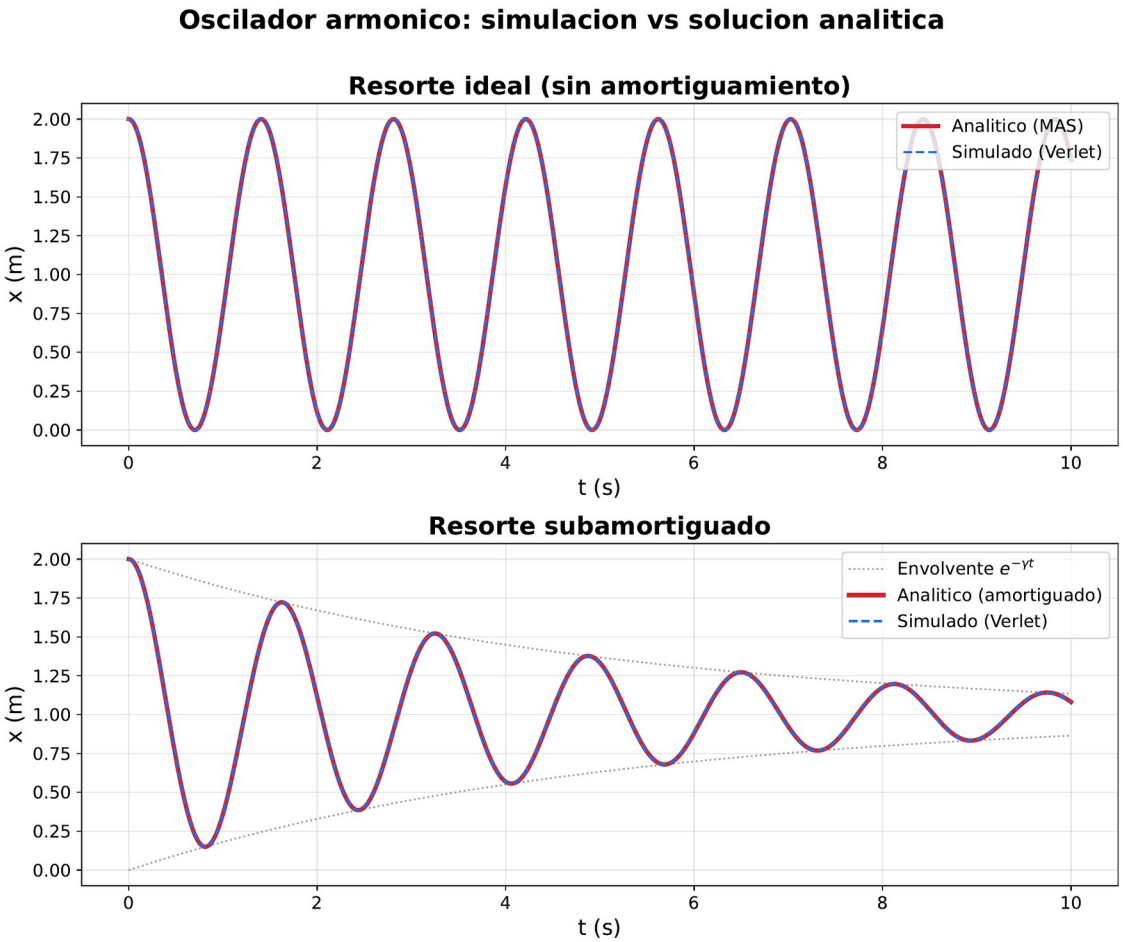


Figura 5. Oscilador armónico ideal y subamortiguado: simulación con Verlet frente a solución analítica.

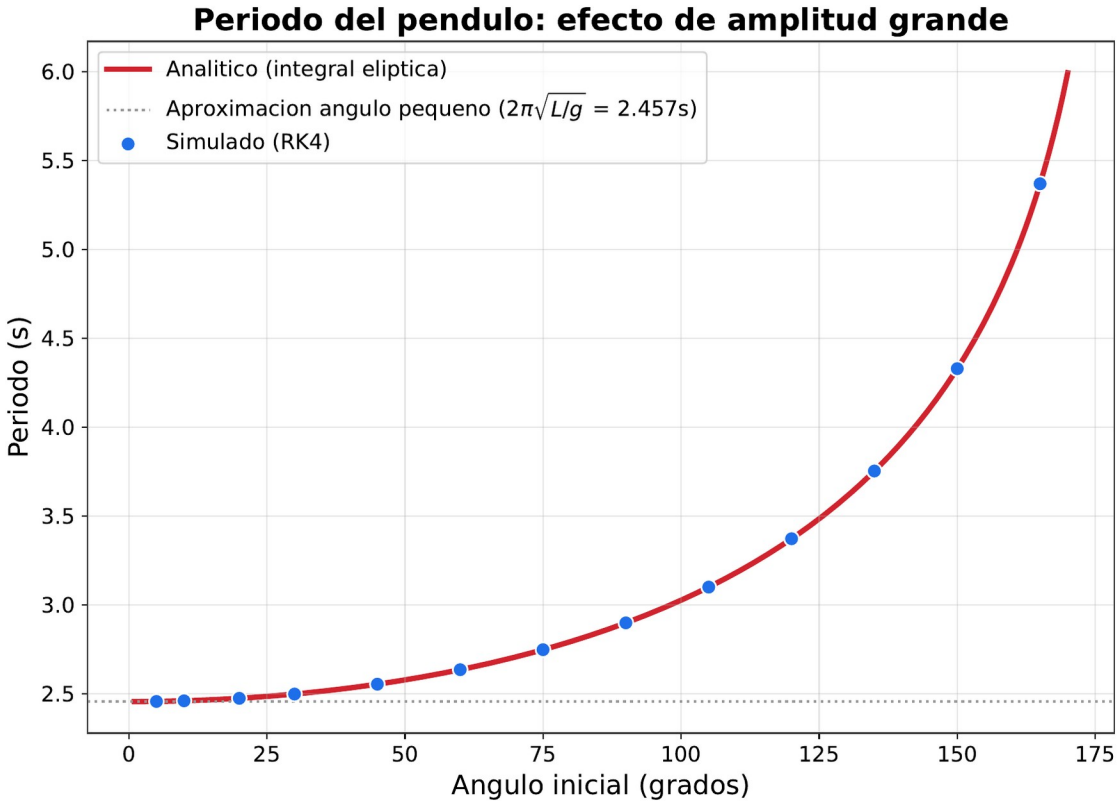


Figura 6. Período del péndulo simple frente a la amplitud, comparado con la integral elíptica.

Pendulo doble: caos determinista con energia conservada

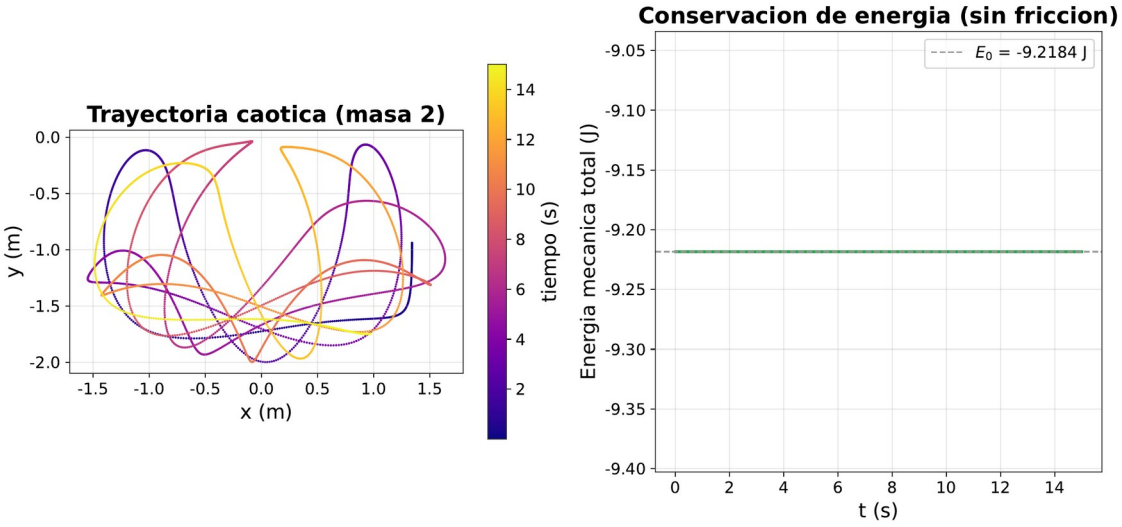


Figura 7. Péndulo doble: trayectoria caótica de la segunda masa y conservación de la energía total.

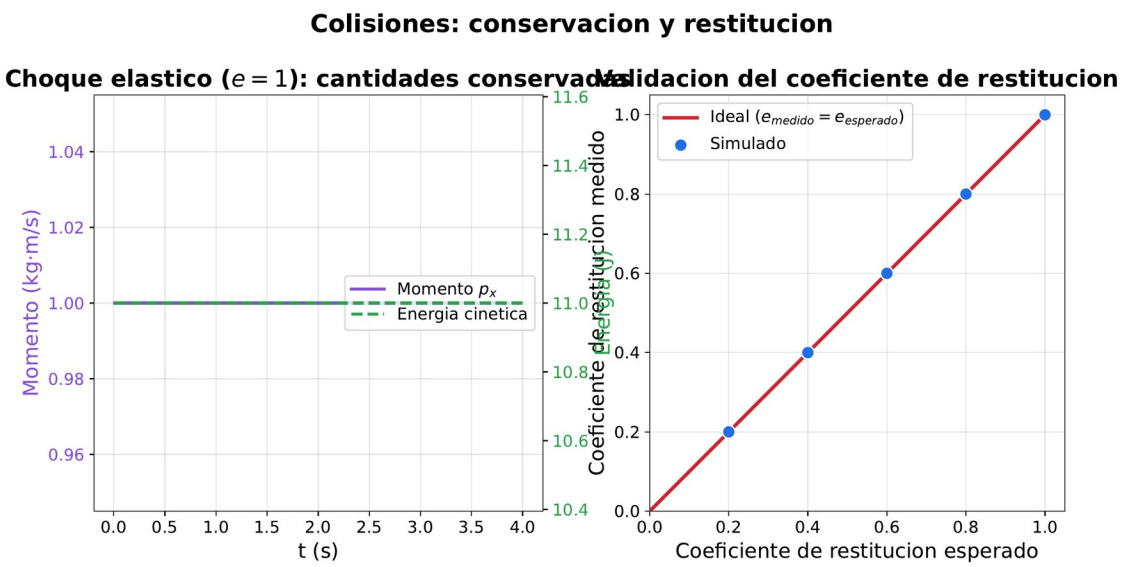


Figura 8. Colisiones: conservación del momento y de la energía, y coeficiente de restitución medido.

Conclusiones

- **Entorno operativo:** se consolidó una primera versión funcional del laboratorio virtual en Python con 5 experimentos de mecánica clásica operativos y dinámicos en tiempo real.
- **Validación física:** los algoritmos numéricos demostraron alta concordancia frente a soluciones analíticas de referencia, con 20 verificaciones automáticas superadas, y preservaron la conservación de energía en los casos aplicables; en el péndulo doble caótico la energía varía 4.3×10^{-7} en términos relativos durante 15 s.
- **Apoyo a la autonomía:** la incorporación del tutor de IA y la interactividad de parámetros facilitan la interpretación física y la estructuración de reportes sin depender de asistencia docente continua.
- **Proyección:** el proyecto se mantiene en desarrollo continuo para integrar nuevos sistemas dinámicos y evaluar su impacto formal en el aprendizaje de los estudiantes.
- Limitaciones y trabajo futuro: el laboratorio aún no se ha evaluado con estudiantes; el tutor depende de un servicio externo, Groq, y de conexión a internet; las simulaciones se limitan a movimiento en 2D y a cinco sistemas. Como trabajo futuro se plantea agregar sistemas como el péndulo forzado, órbitas y fluidos, y desplegar la plataforma en un servidor público.

Bibliografía y páginas web consultadas

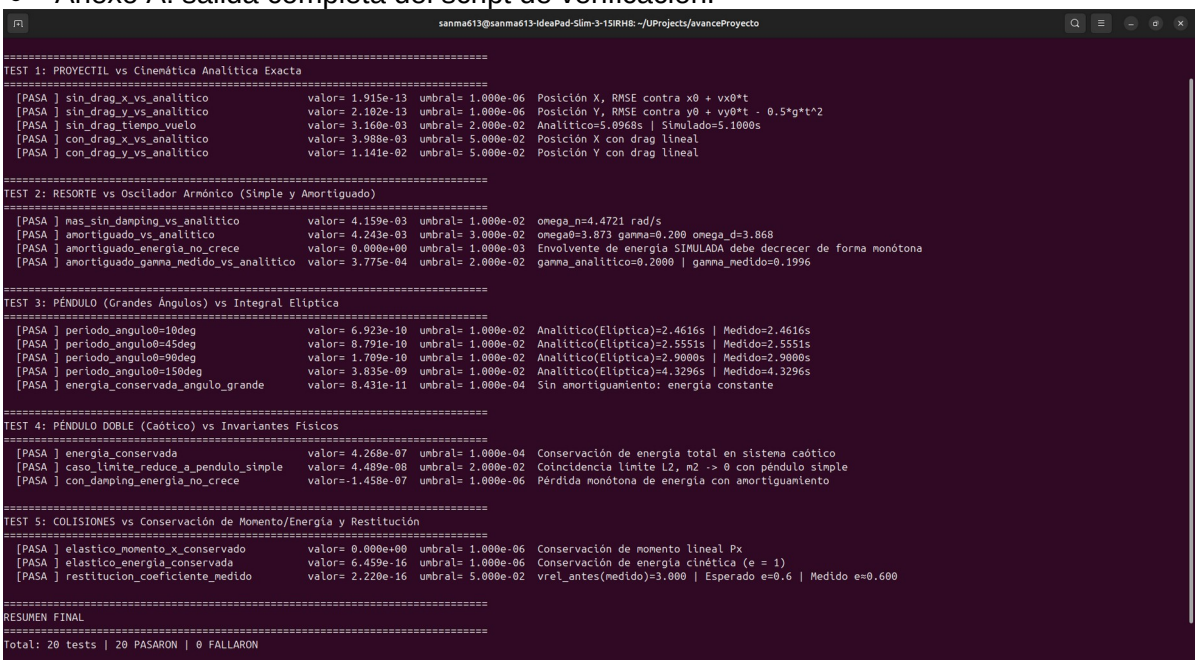
Referencias en formato APA 7.

- Sears, F. W. y Zemansky, M. W. Física universitaria, vol. 1 y 2, 12.ª ed. Addison-Wesley.
- Alonso, M. y Finn, E. Física, vol. 1 y 2. Addison-Wesley.
- Schecker, H. (1993). Learning physics by making models. En Modelling and Computer Simulation in Physics Education (NATO ASI Series F, Vol. 107). Springer.
- Giordano, N. J. y Nakanishi, H. Computational Physics, 2.ª ed. Pearson (2005).
- Python Software Foundation (2024). Python Documentation.
- Ramírez, S. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com/ (Consultado en

octubre de 2026)
- Vercel. (s. f.). Next.js documentation. https://nextjs.org/docs (Consultado en octubre de 2026)
- Groq. (s. f.). GroqCloud documentation. https://console.groq.com/docs (Consultado en octubre de 2026)

Anexos

- Anexo A: salida completa del script de verificación.



- Anexo B: capturas de la interfaz del laboratorio, como la Figura 1.
- Anexo C: repositorio del proyecto, con código fuente y documentación técnica, https://github.com/sanma613/matbiom-lab-virtual-fisica